Gujarat University
Department of Computer Science,
MCA -II 2026 Java Programming
Assignment -2 (UNIT-5,6,8,9,11)

Unit-5 Java Generics

Q-1 Design and implement a Java program using a **generic class Box<T>** that can store and retrieve values of different data types. The program should allow the user to **select the data type at runtime** (Integer, String, or Double) and perform operations on the selected type.

The program must be **menu-driven** and support the following operations:

1. Store a value in the box
2. Display the stored value
3. Exit the program

Based on the user's choice of data type, an appropriate **generic object** should be created, and operations should be performed accordingly.

Q-2 Develop a Java program using **generic classes and methods** to create a flexible data management system. The program should:

- Store different types of data using a **generic container class <T>**
- Use **multiple type parameters <T, U>** for handling paired data (e.g., ID and Name)
- Implement **bounded generics** to perform arithmetic operations only on numeric data
- Use **wildcards** to display and process elements from different collections

The system should provide a menu-driven interface for storing, retrieving, and displaying data.

Unit-6. Comparators and Lambda Expressions

Q-1 Create a java simple program to:

Add student details (id, name, marks)
Display students
Sort by ID (Comparable)
Sort by Name (Comparator)
Sort by Marks (Comparator)
Exit

Q-2 Write a java program that create Class: Student (id, name, marks) and do the following Sort (multilevel):

First by marks
If marks same → sort by name

Q-3 Write a java program that create Class: Player(name, score) and do the following Sort (multilevel):

- Display:
  - Top 3 players
  - Sorted leaderboard

Unit-8 Collection Framework, java.time package, java.util package

Q1. Write a menu-driven Java program to manage student records using List<Student> with attributes (id, name, marks). Perform the following operations:

1. Add student
2. Display all students
3. Search student by name
4. Update student details using index
5. Remove student by index
6. Sort students based on marks
7. Exit

change:

- **Class name (Student → Employee/Product/Book)**
- **Variables (marks → salary/price)**

Q2. Write a Java program to manage orders using List<Order> with attributes (id, itemName, amount). The program should:

1. Add order
2. Display all orders
3. Search order by item name
4. Update order details
5. Remove order by index
6. Sort orders by amount

Q3. Write a Java program to:(HashSet (Duplicate Removal))

- Store student names in a HashSet
- Add duplicate names
- Display all names
- Show that duplicate entries are not stored

Q4. Write a Java program to: LinkedHashSet (Maintain Order

- Store city names in a LinkedHashSet

- Add multiple city names including duplicates
- Display all cities
- Maintain insertion order

Q5. Write a Java program to: TreeSet (Sorted Data)

- Store integer values in a TreeSet
- Add elements
- Display elements in sorted order

Q6. Write a Java program to: HashMap (Phone Directory)

- Store name and phone number using HashMap
- Add multiple entries
- Display all entries
- Search phone number using name

Q7. Write a Java program to: HashMap (Update & Replace)

- Store product ID and name using HashMap
- Add entries
- Replace value of an existing key
- Display updated map

Q8. Write a Java program to: TreeMap (Sorted Key-Value)

- Store employee ID and name using TreeMap
- Add entries
- Display all entries in sorted order of keys

Q9. Write a Java program to: Queue (Ticket System)

- Use Queue<String>
- Add customer names
- Remove customers one by one using FIFO
- Display each removed customer

Q10. Write a Java program to: PriorityQueue (Priority Handling)

- Store integer values in a PriorityQueue
- Add elements
- Display elements in priority order

Q11. Write a Java program to: Deque (Double End Operations)

- Use Deque<Integer>
- Add elements at both ends
- Remove elements from both ends
- Display final deque

Q12. Write a Java program to: Mixed (Set + List)

- Store integers in a List
- Convert the list into a Set to remove duplicates
- Display unique elements

Q13. Write a Java program to simulate a simple Employee Attendance System using the java.time package. When the program runs, record and display the current date and login time of an employee using appropriate classes. Print a message such as "Employee logged in on [date] at [time]".

Q-14 Write a Java program to simulate a simple Customer Registration System using the Scanner class from the java.util package. Accept user input for customer name and age, and display a confirmation message such as "Customer [name], age [age], registered successfully."

Q-15. Write a Java program to simulate a simple OTP (One-Time Password) Generation System using the Random class from the java.util package. Generate a 4-digit or 6-digit random OTP and display it on the screen.

Unit -9 Stream API

Q-1 Write a simple **Java program** using the Java Stream API to perform operations on a list of integers. The program should:

- Display even numbers from the list
- Display odd numbers from the list
- Display square of each number
- Exit the program

The operations must be performed using **Stream methods like filter() and map().**

Q-2 Write a simple **Java program** using the Java Stream API to perform operations on a list of strings.(String Operations) The program should:

- Display names starting with a specific character (e.g., 'S')
- Convert all names to uppercase
- Sort the list of names
- Count total number of names
- Exit the program

Use **Stream operations like filter(), map(), sorted(), and count()**.

Q-3 Write a simple **driven Java program** using the Java Stream API to perform aggregate operations on a list of integers.(aggregate Operations)

The program should:

- Find the sum of all elements
- Find the maximum element
- Store even numbers into a new list
- Exit the program

Use **Stream operations like reduce(), max(), and collect()**

Unit-11 Threads

Q-1 Write a Java program to demonstrate multithreading and inter-thread communication as follows:

- Create two threads:
  - One using **Thread class**
  - One using **Runnable interface**
- Assign names and priorities to both threads and display them using `currentThread()`.
- Implement a **shared resource (buffer)** where:
  - One thread acts as **Producer**
  - One thread acts as **Consumer**
- Use:
  - `synchronized` keyword for thread safety
  - `wait()` and `notify()` for communication
- Display:
  - Thread execution messages
  - Thread states using `isAlive()`
- Demonstrate at least one thread method:
  - `sleep()` or `yield()`

Q-2 Write a Java program to demonstrate thread properties and control methods:

- Create three threads
- Set thread names and priorities
- Use `sleep()` and `yield()`
- Display thread status using `isAlive()`
- Use `join()` to control execution order

Q-3 Write a Java program to demonstrate thread lifecycle and methods:

- Create a thread and display its states
- Use `start()`, `run()`, `sleep()`
- Check thread status using `isAlive()`
- Rename main thread
- Display current thread using `currentThread()`

Q-4 Write a Java program to simulate a **File Download Manager** where multiple files are requested for download. Create threads by **either implementing the Runnable interface OR extending the Thread class**. Allow only a limited number of threads (2 or 3) to download at the same time using synchronization, while others wait. Display messages when a download starts and completes.

Q.5 Write a Java program to simulate a **Bank Account system** where multiple threads perform deposit and withdrawal operations. Create threads by **either implementing the** Runnable **interface OR extending the** Thread **class**. Use synchronized to safely update the balance and prevent withdrawal when the balance is insufficient. Display the updated balance after each transaction.